

# Xinu xinu-rpi4 ユーザーズマニュアル

Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64) 向け Embedded Xinu

Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64) 向けの Embedded Xinu 移植版。同じソースツリーから QEMU virt 機にもクロスビルドできる。本マニュアルは「動かす側」向け — リポジトリの開発ロードマップは README.md を参照のこと。

## 目次

|     |                                           |   |
|-----|-------------------------------------------|---|
| 1   | 対象ターゲットとビルド成果物                            | 1 |
| 1.1 | 機能対応マトリクス (重要)                            | 1 |
| 1.2 | 起動シーケンス (特殊な部分)                           | 2 |
| 2   | 用意するもの                                    | 2 |
| 2.1 | ハードウェア (実機の場合)                            | 2 |
| 2.2 | ホスト側ソフトウェア                                | 2 |
| 3   | ビルド手順                                     | 3 |
| 3.1 | 基本コマンド                                    | 3 |
| 3.2 | ツールチェインの自動検出                              | 3 |
| 3.3 | 主な Make 変数                                | 3 |
| 3.4 | ビルド時の主な定義 (CFLAGS で渡る)                    | 4 |
| 3.5 | リンカスクリプト                                  | 4 |
| 4   | SD カードへの書き込み                              | 4 |
| 4.1 | Pi 4                                      | 4 |
| 4.2 | 手動で焼く場合                                   | 4 |
| 4.3 | SD カードに必要なもの                              | 5 |
| 5   | シリアルコンソールへの接続                             | 5 |
| 6   | シェルコマンドリファレンス                             | 5 |
| 6.1 | procdemo の見どころ                            | 6 |
| 6.2 | pingpong との違い                             | 6 |
| 6.3 | オンデバイス・ツール (ファイルシステム / C JIT / アクタ / LLM) | 7 |
| 6.4 | 仮想メモリ (vmtest / vmdemand)                 | 7 |

|     |                                               |    |
|-----|-----------------------------------------------|----|
| 7   | ネットワーク機能 (Pi 4)                               | 7  |
| 7.1 | Mac から ping する手順 . . . . .                    | 7  |
| 7.2 | DHCP / TCP の現状 . . . . .                      | 8  |
| 7.3 | WiFi (BCM43455) . . . . .                     | 8  |
| 7.4 | 複数 Xinu でのメッシュネットワーク (MANET ad-hoc) . . . . . | 8  |
| 7.5 | HTTP 制御面とリモートシェル . . . . .                    | 9  |
| 8   | QEMU で試す                                      | 10 |
| 9   | config.txt の主な設定                              | 10 |
| 10  | USB / 入力デバイスのサポート状況                           | 11 |
| 11  | トラブルシューティング                                   | 11 |
| 12  | ディレクトリ構成 (利用者目録)                              | 11 |
| 13  | よくある操作レシピ集                                    | 12 |
| 14  | 参考リンク                                         | 12 |
| 15  | ライセンス                                         | 12 |

## 1 対象ターゲットとビルド成果物

| ターゲット | ボード / 機種                     | 出力イメージ          | UART0 base |
|-------|------------------------------|-----------------|------------|
| pi4   | Raspberry Pi 4 (BCM2711)     | kernel8.img     | 0xFE201000 |
| qemu  | QEMU -M virt -cpu cortex-a76 | kernel_virt.img | 0x09000000 |

load address は 0x80000 (Pi 4) / 0x40080000 (QEMU)。2 つのイメージは同一ソースから -D<TARGET> と異なるリンカスクリプトで作り分けている。

### 1.1 機能対応マトリクス (重要)

「全ターゲットで動く」訳ではない。実機 (Pi 4) でしか動かない機能が複数あるので要注意:

| サブシステム           | Pi 4 | QEMU | 備考                               |
|------------------|------|------|----------------------------------|
| PL011 UART0 シェル  | ✓    | ✓    | 全機種で動く土台                         |
| 協調的スケジューラ (S0)   | ✓    | ✓    | procdemo / pingpong              |
| プリエンプティブ (S1)    | ✓    | WIP  | Pi 4 のみ 100 Hz tick 稼働 (ticks)   |
| HDMI フレームバッファ    | ✓    | —    | Pi 4 で実用                         |
| ウィンドウマネージャ       | ✓    | —    | 1280×960 仮想デスクトップは Pi 4 のみ       |
| SD カード + FAT32   | ✓    | —    | EMMC ドライバが BCM2711 向け            |
| Ethernet (GENET) | ✓    | —    | NET-E 完了 (BCM2711 GENET)         |
| USB (キーボード/HID)  | ✓    | —    | VL805 PCIe xHCI 経由 (USB-2.0 ポート) |
| DHCP / TCP       | ×    | —    | コードは在るが dispatch OFF (§7.4)      |

！ソースツリーには `device/genet/`, `device/sd/`, `device/usb/xhci/` 等が同居しており、これらは **Pi 4 (BCM2711)** のレジスタアドレスで初期化される。`-DGENET_BASE` 等は Pi 4 ビルドでのみ定義され、QEMU ビルドでは `undef` になって対応コードがリンクから外れる作り。

## 1.2 起動シーケンス (特殊な部分)

`loader/boot.S` は通常の bare-metal stub よりひと工夫が要る:

1. **Linux ARM64 Image ヘッダが必須。** Pi 4 の EEPROM bootloader はファイル先頭から `0x38` オフセットに "ARM\64" マジックが無いとカーネルへジャンプしないケースがある。`boot.S` の `_start:` 直下に 64 バイトのヘッダを配置。無いと **HDMI に虹色が出たまま固まる**。
2. **MPIDR\_EL1 によるコア固定。** boot core 以外 (core1/2/3) を wfe ループに park。
3. **スタック設定。** `sp = _start (= 0x800000)`。
4. **DTB アドレス退避。** ファームウェアから `x0` に渡される DTB の物理アドレスを BSS clear の間退避し `.data dtb_addr` に格納。
5. **BSS クリア → kernel\_main。** `kernel_main` から戻れば secondary と同じ wfe パークへ落ちる。

EL のドロップ (EL2 → EL1) は自分で実装していない (QEMU も同じ前提)。`armstub` や `kernel_old=1` を入れると状況が変わる。

## 2 用意するもの

### 2.1 ハードウェア (実機の場合)

- Raspberry Pi 4 (BCM2711)
- microSD カード (FAT32 で bootfs パーティション)。一番簡単なのは Raspberry Pi OS を焼き、後で `kernel8.img` と `config.txt` を上書き
- USB-シリアル変換ケーブル (3.3V、115200 8N1) — ヘッダピン 8 (TXD → GPIO14) / 10 (RXD → GPIO15) / 6 (GND)
- ネットワーク機能を使う場合: イーサネットケーブル

## 2.2 ホスト側ソフトウェア

macOS / Linux と AArch64 クロスツールチェーン (どちらか):

```
brew install aarch64-elf-gcc      # Homebrew (推奨)
brew install --cask gcc-arm-embedded  # ARM 公式
```

QEMU で動かすなら `qemu-system-aarch64` (`brew install qemu`)、およびターミナルシリアルクライアント (`screen`、`minicom`、`picocom`)。

## 3 ビルド手順

ビルドは必ず `compile/` ディレクトリで行うこと (Makefile が `VPATH` で `../loader/`、`../device/...` を集める作り)。

### 3.1 基本コマンド

```
cd /Users/kodamay/projects/xinu-rpi4/compile

make pi4      # -> compile/kernel8.img      (Raspberry Pi 4 実機)
make qemu     # -> compile/kernel_virt.img  (QEMU virt)
make          # = make all = pi4 + qemu
make clean    # オブジェクト・全 .img を削除
```

各ターゲットのオブジェクトは別ツリー (`obj/pi4/`、`obj/qemu/`) に分離され、続けて別ターゲットをビルドしても再コンパイルは差分のみ。

### 3.2 ツールチェーンの自動検出

`$(GCCPATH)/bin/aarch64-elf-gcc` → `aarch64-none-elf-gcc` の順に探す。既定 `GCCPATH` は `/opt/homebrew`。別の場所なら:

```
make pi4 GCCPATH=$HOME/aarch64/arm-gnu-toolchain-...-aarch64-none-elf
make pi4 CROSS=aarch64-linux-gnu-
```

### 3.3 主な Make 変数

| 変数      | 既定値               | 用途                       |
|---------|-------------------|--------------------------|
| GCCPATH | /opt/homebrew     | クロスコンパイラのインストール先         |
| CROSS   | (自動検出)            | プレフィックス (aarch64-elf- 等) |
| SDCARD  | /Volumes          | make install_* のマウント親    |
| DEST    | \$(SDCARD)/bootfs | コピー先パーティション              |

### 3.4 ビルド時の主な定義 (CFLAGS で渡る)

| マクロ         | Pi 4            | QEMU virt         |
|-------------|-----------------|-------------------|
| -mcpu       | cortex-a72      | cortex-a76        |
| UART0_BASE  | 0xFE201000      | 0x09000000        |
| MBOX_BASE   | 0xFE00B880      | (未定義)             |
| SD_BASE     | 0xFE340000      | —                 |
| USB_BASE    | 0xFE980000      | —                 |
| GIC_BASE    | 0xFF840000      | —                 |
| PCIE_BASE   | 0xFD500000      | —                 |
| GENET_BASE  | 0xFD580000      | —                 |
| HEAP_END    | 0x40000000 (1G) | 0x50000000        |
| SKIP_MBOX   | —               | 定義 (簡略化)          |
| KERNEL_NAME | "kernel8.img"   | "kernel_virt.img" |
| BOARD_NAME  | "Pi4"           | "virt"            |
| SOC_NAME    | "BCM2711"       | "QEMU"            |

共通 CFLAGS: -Wall -O2 -ffreestanding -nostdinc -nostdlib -nostartfiles -mgeneral-regs-only。大半のコードで FPU/NEON は使わず、cc.c / llm.c のみ FP 有効でビルドする。

### 3.5 リンカスクリプト

| スクリプト                | 対象        | エントリ・ロードアドレス |
|----------------------|-----------|--------------|
| compile/link.ld      | Pi 4      | 0x80000      |
| compile/link_virt.ld | QEMU virt | 0x40080000   |

どちらも .text.boot を先頭に置き、\_\_bss\_start / \_\_bss\_size (8-byte 単位) をエクスポートする leex 風の構成。

## 4 SD カードへの書き込み

### 4.1 Pi 4

```
cd compile
make install_pi4 SDCARD=/Volumes
```

```
# -> /Volumes/bootfs に kernel8.img と sdcard/config_pi4.txt をコピー
```

make install は install\_pi4 のエイリアス。

## 4.2 手動で焼く場合

```
diskutil mount /dev/disk4s1          # -> /Volumes/bootfs
cp compile/kernel8.img /Volumes/bootfs/
cp sdcard/config_pi4.txt /Volumes/bootfs/config.txt
sync
diskutil eject /Volumes/bootfs
```

! kernel8.img の md5 が SD カード上のものと一致するか必ず確認すること。古いイメージで起動して混乱するのは「あるある」。

## 4.3 SD カードに必要なもの

kernel8.img と config.txt だけでは起動しない。同じ FAT32 パーティションに Raspberry Pi OS の bootcode.bin, start4.elf, fixup4.dat ほかにファームウェア blob 一式も必要。Pi OS を焼いてから kernel8.img と config.txt だけ上書きするのが最速。

## 5 シリアルコンソールへの接続

```
screen /dev/tty.usbserial-XXXX 115200      # Mac
minicom -b 115200 -o -D /dev/tty.usbserial-XXXX  # Mac
sudo screen /dev/ttyUSB0 115200            # Linux
```

screen を終了するときは Ctrl-a k → y。電源投入後、約 5 秒以内に次のバナーが出る:

```
=====
Xinu Pi4 hello (AArch64, BCM2711, kernel8.img)
PL011 UART0 @ 0xFE201000, 115200 8N1
bootstrap: leex-style stub + xinu-rpi4 main
=====

Round 1 phase B/U done -- entering interactive shell.
type `help` for the command list.
xinu-pi4$ _
```

## 6 シェルコマンドリファレンス

help または ? で常に最新の一覧が見られる。現在のコマンド一覧:

| コマンド                                                     | 用途                                                    |
|----------------------------------------------------------|-------------------------------------------------------|
| help / ?                                                 | 登録コマンド一覧を表示                                           |
| echo <words>                                             | 引数をそのままエコーバック                                         |
| hello                                                    | 動作確認用の挨拶                                              |
| mem                                                      | __bss_start / __bss_end / _end を表示                    |
| peek <hex_addr>                                          | 32-bit MMIO ワードを読む (例: peek 0xfe201018)               |
| uptime                                                   | 生の CNTPCT_EL0 (汎用タイマカウンタ)                             |
| ticks                                                    | 100 Hz タイマ tick の累積数 (S1)                             |
| ps                                                       | コア / EL ステータス簡易表示                                     |
| halt                                                     | DAIF マスク + PSCI SYSTEM_OFF (QEMU はクリーン終了)             |
| reboot                                                   | スタブ — 電源再投入待ちでスピン                                     |
| pwd ls cd mkdir rmdir touch cat write edit rm cp mv tree | RAM 上のファイルシステム (揮発性。§6.3)                             |
| cc <file.c>                                              | C をオンデバイスでコンパイル&実行 (JIT → AArch64。§6.3)               |
| aload / amsg                                             | 常駐 AIPL アクタのロード / メッセージ送信 (§6.3)                      |
| actordemo / selectdemo                                   | アクタ ping-pong / 名前付き選択受信 (§6.3)                       |
| vmtest / vmdemand                                        | VA≠PA リマップ / デマンドページング仮想メモリ (§6.4)                    |
| llm [prompt]                                             | 組込み LLM でテキスト生成 (§6.3)                                |
| preempt                                                  | タイマ駆動プリエンプティブ RR のデモ                                  |
| pingpong [N]                                             | AIPL 風 2 アクター協調 PingPong (N=1..50)                    |
| procdemo [N]                                             | 実 ctxsw による 2 プロセスデモ (N=1..30)                        |
| usb                                                      | xHCI / DWC2 USB 診断 (Pi 4 のみ)                          |
| wifi ...                                                 | WiFi + メッシュ: probe/scan/up/adhoc/aodv/... (§7.5–§7.6) |
| rxstat / tcpstat                                         | RX リング / TCP listener カウンタ                            |
| pan <dx> <dy> view autopan [on off]                      | ウィンドウマネージャのビューポート操作                                   |

## 6.1 procdemo の見どころ

```
xinu-pi4$ procdemo 3
procdemo: created pid=1 (ping) and pid=2 (pong), iters=3
  [Ping pid=1] tick 1
  [Pong pid=2] tock 1
  [Ping pid=1] tick 2
  [Pong pid=2] tock 2
  [Ping pid=1] tick 3
  [Pong pid=2] tock 3
  [Ping pid=1] exit at iter 3
  [Pong pid=2] exit at iter 3
procdemo: both processes exited; back in shell.
```

pid=N はグローバル currpид から実行時に読み出した値なので、列方向の交互パターン自体がスケジューラが本当に切替えている証拠になる。スタックは proc\_exit では回収していない (S1 で IRQ tick 経由で回収予定)。

## 6.2 pingpong との違い

| 観点   | pingpong            | procdemo                     |
|------|---------------------|------------------------------|
| アクター | 静的 2 つ (Ping, Pong) | proc_create した実プロセス          |
| スイッチ | 単一スタック上のディスパッチ      | ctxsw.S による実 callee-saved 保存 |
| 停止条件 | 両者の inbox が空        | 両者が proc_exit()              |

## 6.3 オンデバイス・ツール (ファイルシステム / C JIT / アクタ / LLM)

上記のデモに加え、シェルには自己完結したツールチェーンが載っており、ボード上でコードを書いて・コンパイルして・実行できます。

- **RAM 上のファイルシステム** — `pwd ls cd mkdir rmdir touch cat write edit rm cp mv tree` がメモリ上の小さなツリーを操作 (揮発性。再起動で消える)。
- **C JIT (cc)** — `cc <file.c>` が C のサブセットをネイティブ AArch64 にコンパイルしてその場で実行。同じコンパイラは HTTP の `POST /compile` (本文 = C ソース) から使えます。
- **AIPL アクタ** — `aload <file.c>` で常駐アクタをロード、`amsg <actor> <method> [arg]` で送信。`actordemo` は 2 アクターの ping-pong を実 Xinu プロセスとして実行、`selectdemo` はガード付き受信を示します。HTTP の `/actor`・`/send`・`/gc` がアクター一覧・送信・アクタプール GC を公開。
- **組込み LLM (llm)** — 小さな transformer がイメージに焼き込まれており、`llm [prompt]` でオンデバイス生成 (HTTP の `/chat` でも)。

## 6.4 仮想メモリ (vmtest / vmdemand)

カーネルは MMU 有効 (identity map) + VA `0x80000000..0x80400000` (4 MiB, 512 フレームプール) のデマンドページング窓で動作します:

- **vmtest** — 同じ物理ページを別の仮想アドレスにマップして変換を実証 ( $VA \neq PA$ )。
- **vmdemand** — デマンド窓の 64 ページに触れる。初回は `0x40` 回のページフォルト (1 ページ 1 回) でリードバック OK、2 回目は `0x0` フォルト (既にマップ済み)。フォルトカウンタは `/fault` で確認。

ローカルでもリモートでも実行可: 例 `curl "http://192.168.3.100/shell?cmd=vmdemand"`。

## 7 ネットワーク機能 (Pi 4)

ネットワークドライバは Pi 4 GENET (BCM2711) で動作する。

**動作する機能:** ARP request/reply、ICMP echo (ping)、静的 IP/MAC `192.168.3.100 / d8:3a:dd:a7:fd:bf` (loader/main.c:707 付近で設定)、生 broadcast/unicast 送信、RX リング 16-slot。

## 7.1 Mac から ping する手順

```
sudo arp -s 192.168.3.100 d8:3a:dd:a7:fd:bf
ping 192.168.3.100
```

RTT は約 900ms (ウィンドウマネージャの tick 律速)。シェルからは `rxstat / ticks` で状況確認。

## 7.2 DHCP / TCP の現状

`system/dhcp_client.c` と `system/tcp_server.c` はツリーに含まれるが、`rx_tick` からの dispatch を **OFF** にしている: 連続 broadcast TX で GENET TX ring が劣化し ICMP が止まる、また `tcp_handle_packet()` を `rx_tick` から呼ぶと以降の ICMP echo reply が止まるため。再有効化は今後のスプリント (NET-G bisect) で対応予定。

## 7.3 WiFi (BCM43455)

Pi 4 はオンボードの BCM43455 WiFi チップを持ちます (上記の有線 GENET とは別系統)。操作はすべてシリアルシェル (`xinu-pi4$`) から行います: ファームウェアを起こし、スキャンし、接続します。**起動時に自動接続はしません。**

| コマンド                                                                                  | 説明                                          |
|---------------------------------------------------------------------------------------|---------------------------------------------|
| <code>wifi probe</code>                                                               | M0/M1 bring-up: チップにファームウェアを DL (起動ごとに 1 回) |
| <code>wifi scan</code>                                                                | 近隣 AP を escan                               |
| <code>wifi up &lt;ssid&gt; &lt;pass&gt;</code>                                        | join (WPA2-PSK) + DHCP。常駐 ARP/ICMP 応答を開始    |
| <code>wifi join &lt;ssid&gt; &lt;pass&gt;</code>                                      | join のみ (DHCP なし)                           |
| <code>wifi dhcp</code>                                                                | DHCP リース取得                                  |
| <code>wifi off</code>                                                                 | 無線停止 / 切断                                   |
| <code>wifi ping &lt;a.b.c.d&gt; [count]</code>                                        | ICMP echo クライアント                            |
| <code>wifi serve [secs]</code>                                                        | ARP/ICMP 応答を N 秒間実行 (既定 30)                 |
| <code>wifi resolve &lt;host&gt; / wifi web &lt;host&gt;</code>                        | DNS / DNS + HTTP GET                        |
| <code>wifi ntp [a.b.c.d]</code>                                                       | NTP 時刻クライアント                                |
| <code>wifi tftp &lt;ip&gt; &lt;file&gt; / wifi netboot &lt;ip&gt; &lt;file&gt;</code> | TFTP 取得 / 取得 + チェインロード                      |

```
xinu-pi4$ wifi probe          # ファームウェア DL (起動ごとに 1 回)
xinu-pi4$ wifi scan           # 周囲の AP を確認
xinu-pi4$ wifi up MyHome-5G mypassword
wifi up: connected; ARP/ICMP responder is now persistent.
xinu-pi4$ wifi ping 8.8.8.8
```

再起動後は自動再接続しません。起動のたびに `wifi probe` + `wifi up` を実行してください。`wifi netboot <ip> <file>` は WiFi 越しにカーネルを取得してチェインロードします (有線 chainload と同様、SD 交換なしの更新経路)。

## 7.4 複数 Xinu でのメッシュネットワーク (MANET ad-hoc)

複数の Pi 4 (および Pi 3) Xinu ボードを **アクセスポイント無し** で直接つなぎ、ピアツーピアのメッシュを組めます。802.11 の IBSS (ad-hoc) モード + オンデマンドの AODV ルーティングを使います (ドローン HIL デモの Pi 4 + Pi 3 ノードと同じ MANET スタック)。各ノードは同じ ad-hoc セルに、別々のノード番号で参加します:

```
xinu-pi4$ wifi probe # 起動ごとに 1 回
xinu-pi4$ wifi adhoc <cell-ssid> [ch] [node]
```

- <cell-ssid> — ad-hoc セル名。全ノードで同じ名前・同じチャンネルにすること。
- [ch] — 802.11 チャンネル (既定 6)。
- [node] — 自分のノード番号 (既定 1)。静的 IP 10.0.0.<node>/24 を得ます。ノードごとに別の番号を与えること。

例 — チャンネル 6 の 3 ノードセル:

```
xinu-pi4$ wifi adhoc mesh1 6 1 # -> 10.0.0.1
xinu-pi4$ wifi adhoc mesh1 6 2 # -> 10.0.0.2
xinu-pi4$ wifi adhoc mesh1 6 3 # -> 10.0.0.3
```

電波の届く範囲のノードは 10.0.0.x で直接通信できます (wifi ping 10.0.0.2)。

**マルチホップ (AODV)**。宛先が直接届かないとき、wifi aodv <ip> でオンデマンドに経路探索します:

```
xinu-pi4$ wifi aodv 10.0.0.3
[wifi] === AODV discover 10.0.0.3 ===
[wifi] aodv: RREQ id=1 for 10.0.0.3
[wifi] *** AODV route: 10.0.0.3 via 10.0.0.2, 2 hop ***
```

最小 AODV モジュール (M13, RFC 3561 コア) が RREQ をブロードキャスト (UDP port 654) し、範囲内の各ノードが中継して逆経路を記録、宛先が RREP を返して順経路を確立します。各ノードはピアのために自動で中継するので、直接届かない宛先も中間ノードが転送します (経路表は最大 16 エントリ)。IBSS/ad-hoc はインフラ (wifi up) モードとは独立です (AP に繋がっている場合は先に wifi off)。再起動後は復元しないので、各ノードで wifi probe + wifi adhoc を再実行してください。

## 7.5 HTTP 制御面とリモートシェル

system/tcp\_server.c は有線インタフェース上で HTTP サーバ (既定ポート 80、192.168.3.100) を動かす、ボードをネットワーク越しに丸ごと操作できます:

| ルート                                            | 内容                             |
|------------------------------------------------|--------------------------------|
| GET /shell?cmd=<cmd>                           | 任意のシェルコマンドを実行し出力を返す (stdin なし) |
| POST /compile                                  | C を JIT コンパイル&実行 (本文 = ソース)    |
| GET /chat                                      | オンデバイス LLM チャット                |
| GET /actor /send /gc /jitstats                 | アクター一覧 / 送信 / GC / JIT カウンタ    |
| GET /usbdiag /pcie-init /pcie-enum /xhci-reset | USB / PCIe bring-up 診断         |
| GET /fault /mmio-read /mmio-write /mmio-sweep  | フォルトカウンタ + 生 MMIO peek/poke    |
| POST /reboot                                   | BCM2711 watchdog リセット          |
| POST /chainload                                | 新カーネルをアップロードして起動 (SD 交換なし)     |

実行時診断にこれら HTTP カウンタを使うのは、uart\_puts が HTTP ワークコンテキストでデッドロックするためです (bring-up ログはブート時にシリアルへ)。

**SD 交換なしのカーネル更新 (chainload)**。POST /chainload は新カーネルをアップロードして RAM 上でジャンプします。アップロードはヘルパスクリプトが包みます:

```
python3 tools/remote_chainload.py 192.168.3.100 compile/kernel8.img
```

不良イメージでも電源を入れ直すだけで戻ります (SD は無傷) ので開発ループが速くなります。アップロードには HTTP サーバが応答可能であることが必要です。

## 8 QEMU で試す

```
cd compile
make qemu          # インタラクティブ起動。Ctrl-A X で抜ける
make qemu-smoke    # 既定のスクリプトを流し込み qemu-smoke.log に保存
```

QEMU 上では: MIDR\_EL1 が 0x414fd0b1 (QEMU が公開する Cortex-A76 の part 番号。実機 Pi 4 では Cortex-A72)、halt は PSCI SYSTEM\_OFF が virt でハンドルされクリーン終了、ネットワーク・USB・SD・HDMI 出力は無し。

## 9 config.txt の主な設定

sdcard/config\_pi4.txt (make install\_pi4 がカード上の config.txt へコピー) の重要項目:

| 行                        | 意味                                  |
|--------------------------|-------------------------------------|
| arm_64bit=1              | AArch64 で起動                         |
| kernel=kernel8.img       | Pi 4 用カーネル名                         |
| kernel_address=0x80000   | カーネルロードアドレス                         |
| enable_uart=1            | UART を有効化                           |
| uart_2ndstage=1          | bootloader 第 2 段の UART ログを出す        |
| dtparam=uart0=on         | UART0 を GPIO14/15 にルーティング           |
| init_uart_clock=48000000 | PL011 基準クロックを 48 MHz に固定 (115200 用) |
| hdmi_force_hotplug=1     | HDMI ホットプラグを強制 ON                   |
| framebuffer_width=640    | ファームウェア確保 FB の幅                     |
| framebuffer_height=480   | FB の高さ                              |
| framebuffer_depth=32     | FB の bpp                            |

KMS overlay (dtoverlay=vc4-kms-v3d) を残してあるが、Xinu 側はファームウェアが用意した simple-framebuffer を mailbox 経由で借りる構成。

## 10 USB / 入力デバイスのサポート状況

Pi 4 のフルサイズ USB-A ポートは **VL805 (PCIe xHCI)** 経由でぶら下がっており、xHCI ドライバを自作して USB キーボード・マウスを動かしている。

- **USB-2.0 (黒) ポートに挿すこと。**USB-2.0 ハブ (Genesys、TT 動作) 経由なので LS の割り込み IN split transaction が届く。USB-3.0 (青) は VIA USB3 ハブ側に入り、周期転送が届かず動かない。
- **usb** コマンドは DWC2 (USB-C OTG) HCD の診断ダンプ専用。
- VL805 のファームウェアは SD ブート時 PCIe RC がリセットのままになるため、**NOTIFY\_XHCI\_RESET** mailbox tag (0x00030058, devid 0x100000) を boot 時に発行して xHCI を起こしている (詳細は **NEXT\_SESSION\_PI4.md**)。

| 機種   | 入力経路                     | 出力経路                 |
|------|--------------------------|----------------------|
| Pi 4 | USB-A キーボード / UART0 シリアル | UART0 シリアル + HDMI FB |
| QEMU | qemu のシリアル stdio         | qemu のシリアル stdio     |

## 11 トラブルシューティング

| 症状                      | 確認ポイント                            |
|-------------------------|-----------------------------------|
| シリアルに何も出ない              | screen のデバイス名と速度、TX/RX 入れ違い、GND   |
| バナーは出るがすぐ止まる            | kernel8.img の md5 が古い。再ビルド + 再コピー |
| キー入力か echo されない         | ターミナルの local echo は OFF が正しい      |
| peek で 0xFFFFFFFF       | そのアドレス帯が未マップ。MMU は flat ID        |
| USB キーボード/マウスが効かない      | USB-2.0 (黒) ポートに挿しているか (§10)      |
| ping が通らない (Pi 4)       | sudo arp -s ... 済か、ケーブル、rxstat 増分 |
| make qemu が失敗           | qemu-system-aarch64 のバージョン (≥11)  |
| make install permission | 内部で sudo cp するためパスワード入力           |
| 起動後すぐ reboot ループ        | config.txt の kernel= 行と実ファイル名     |
| HDMI に虹色が残る             | ARM64 Image ヘッダ欠落の可能性。再ビルド        |

ハマったときの定番動作:

```
cd compile && make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
# SD カード抜き挿し → Pi の電源再投入
```

## 12 ディレクトリ構成 (利用者目線)

```
xinu-rpi4/
├── compile/                # ビルドはここで `make`
│   ├── Makefile
│   ├── kernel8.img        # Pi 4 用 (make pi4 で生成)
│   └── kernel_virt.img    # QEMU 用 (make qemu で生成)
├── sdcard/
│   └── config_pi4.txt      # Pi 4 用 (install_pi4 がコピー)
├── README.md              # 開発者向けロードマップ + 内部実装
├── USERS_MANUAL_JA.md     # 本マニュアルの Markdown 版
├── USERS_MANUAL_EN.md    # 英語版
└── NEXT_SESSION_PI4.md   # 進行中作業のハンドオフメモ
```

## 13 よくある操作レシピ集

ビルドから Pi 4 で起動するまで一気通貫:

```
cd /Users/kodamay/projects/xinu-rpi4/compile
make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
```

```
screen /dev/tty.usbserial-XXXX 115200
```

コマンドを手早く確かめる (QEMU):

```
cd compile && make qemu
xinu-pi4$ procdemo 5
xinu-pi4$ pingpong 3
xinu-pi4$ halt          # Ctrl-A X で QEMU から脱出
```

## 14 参考リンク

- 上流 Xinu (32-bit): <https://github.com/yaskodama/xinu-rpi>
- AArch64 boot stub の元: <https://github.com/radlyeel/leex>
- シェル centry パターンの元: <https://github.com/davidxyz/xinuPi>
- 開発ロードマップ (Round 1 計画書): abclcp-project リポジトリの [aice-pi-evolution/experiments/](#) 配下

## 15 ライセンス

上流 Xinu と leex の BSD 系ライセンスを継承。詳細は LICENSE を参照 (整備中)。